Azure NetApp Files AI Enablement Training Accelerators

Technical Blueprint & Implementation Guide

Version: 1.0
Prepared for: Microsoft Internal AI Enablement Training Session
Target Audience: Sales, Architects, Technical Community (100+ attendees)
Author: D
Date: January 2026

Executive Summary

This document provides the technical blueprint for three production-ready GitHub accelerator repositories that demonstrate Azure NetApp Files (ANF) integration with Azure's AI and analytics ecosystem. These accelerators showcase how the new **Object REST API** (S3-compatible, currently in public preview) enables seamless data access for AI workloads without data movement.

Accelerator Repositories

Repo	Scenario	Key Technologies
anf-onelake-fabric-agents	ANF → OneLake → Fabric Data Agents	Object REST API, S3 Shortcuts, Fabric Lakehouse
anf-ai-search-rag	ANF → Azure AI Search → RAG Chat	Object REST API, Vectorization, Azure OpenAI
anf-foundry-selfops	Foundry Agents → ANF Management	Control Plane REST API, Function Calling

---

Part 1: Technical Foundations

1.1 Azure NetApp Files Object REST API

Status: Public Preview (as of October 2025)
Documentation: Microsoft Learn - Object REST API

Core Capabilities

```
┌─────────────────────────────────────────────────────────────────────┐
│                    Azure NetApp Files Volume                        │
│                                                                     │
│  ┌─────────────────────────────────────────────────────────────────┐ │
│  │                    File/Object Duality                          │ │
│  │                                                                 │ │
│  │  /enterprise-data/          ← NFS/SMB Access                    │ │
│  │  ├── documents/                                                 │ │
│  │  ├── datasets/                                                  │ │
│  │  └── models/                                                    │ │
│  │                                                                 │ │
│  │  s3://anf-bucket/           ← Object REST API Access            │ │
│  │  ├── documents/                                                 │ │
│  │  ├── datasets/                                                  │ │
│  │  └── models/                                                    │ │
│  │                                                                 │ │
│  │  SAME DATA - DUAL ACCESS PROTOCOLS                              │ │
│  └─────────────────────────────────────────────────────────────────┘ │
└─────────────────────────────────────────────────────────────────────┘
```

Supported S3 Operations

Operation	Status	Use Case
ListBucket	✅	Directory enumeration
ListObjects	✅	File listing
GetObject	✅	Read files for AI processing
PutObject	✅	Write AI outputs
DeleteObject	✅	Cleanup operations
HeadObject	✅	Metadata queries

Verified Integrations (from Microsoft Documentation)

- **Azure AI Search** - Index directly from ANF buckets
- **Azure AI Foundry** - Document processing for AI
- **Azure Databricks** - Spark access to ANF data
- **Microsoft Fabric / OneLake** - S3-compatible shortcuts

- **Azure Synapse Analytics** - Data lake integration
- **Azure OpenAI** - Document vectorization for RAG

Prerequisites for Object REST API

1. **Waitlist enrollment** (approximately 1 week activation)
2. **PEM-formatted SSL certificate** (Azure Key Vault or self-generated)
3. **Bucket configuration** with UID/GID for access control
4. **Access credentials** (Access Key + Secret Key)

Configuration Example

```bash
# Bucket endpoint format
https://<your-bucket-endpoint-IP-or-FQDN>

# AWS CLI access (always use us-east-1 region)
aws s3 ls --endpoint-url https://10.0.0.5 s3://enterprise-bucket/ \
  --region us-east-1
```

1.2 Azure NetApp Files Control Plane REST API

SDK: `azure-mgmt-netapp` (v14.0.1 as of November 2025)
API Version: 2024-07-01 (stable) / 2025-09-01 (latest)

Management Operations for Agentic SelfOps

Operation	Endpoint	Use Case
Create Snapshot	`PUT .../snapshots/{name}`	Pre-change safety
List Snapshots	`GET .../snapshots`	Recovery points
Delete Snapshot	`DELETE .../snapshots/{name}`	Cleanup
Restore Snapshot	`POST .../revert`	Rollback
Create Clone	`POST .../volumes` (from snapshot)	A/B testing
Resize Volume	`PATCH .../volumes/{name}`	Capacity scaling
Change Throughput	`PATCH .../volumes/{name}`	Performance tuning

Python SDK Pattern

```python
from azure.identity import DefaultAzureCredential
from azure.mgmt.netapp import NetAppManagementClient

# Initialize client
credential = DefaultAzureCredential()
client = NetAppManagementClient(credential, subscription_id)

# Create snapshot (async operation)
poller = client.snapshots.begin_create(
    resource_group_name="myRG",
    account_name="account1",
    pool_name="pool1",
    volume_name="volume1",
    snapshot_name="agent-snapshot-001",
    body={"location": "eastus"}
)
result = poller.result()  # Wait for completion
```

Part 2: Scenario 1 - ANF + OneLake + Fabric Data Agents

2.1 Architecture Overview

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                    SCENARIO 1: Analytics Pipeline                           │
├─────────────────────────────────────────────────────────────────────────────┤
│                                                                             │
│  ┌──────────────────┐                                                       │
│  │ Data Engineers   │                                                       │
│  │ (NFS/SMB)        │                                                       │
│  └──────────────────┘                                                       │
│          │ Write data via traditional protocols                             │
│          ▼                                                                  │
│  ┌──────────────────────────────────────────────────────────────────────┐   │
│  │   Azure NetApp Files Volume                                          │   │
│  │                                                                      │   │
│  │ /sales-data/                                                         │   │
│  │ ├── transactions.parquet                                             │   │
```

```
│  ├── customers.delta/          │                    │
│  └── products.csv              │                    │
│                                │                    │
│  Object REST API Bucket:       │                    │
│  s3://sales-analytics-bucket   │                    │
└────────────────┬───────────────────────────────────┘
                 │ S3-compatible access
                 ▼
┌──────────────────────────────────────────────────┐
│  On-Premises Data Gateway      │◄── Required for VNet access
│  (VM in same VNet as ANF)      │
└────────────────┬───────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────────────────────┐
│  Microsoft OneLake             │                    │
│                                │                    │
│  Workspace: Sales-Analytics    │                    │
│  └── Lakehouse: sales_lakehouse │                    │
│     └── S3 Shortcut: anf_sales │◄── Points to ANF bucket
│        └── transactions        │                    │
│        └── customers           │                    │
│        └── products            │                    │
└────────────────┬───────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────────────────────┐
│  Microsoft Fabric Data Agent   │                    │
│                                │                    │
│  Data Sources:                 │                    │
│  - sales_lakehouse (via shortcut) │                    │
│                                │                    │
│  Natural Language Query:       │                    │
│  "What were top 10 products by │                    │
│   revenue last quarter?"       │                    │
│                                │                    │
│  → Generates SQL automatically │                    │
│  → Executes against Lakehouse  │                    │
│  → Returns formatted answer    │                    │
└────────────────┬───────────────────────────────────┘
                 │
```

2.2 Technical Viability Assessment

Component	Status	Documentation Reference
ANF Object REST API	✅ Public Preview	Configure Object REST API
OneLake S3 Shortcut	✅ GA	OneLake Shortcuts
On-Premises Data Gateway	✅ GA (Required)	Connect OneLake to ANF
Fabric Data Agent	⚠️ Preview	Data Agent Creation

Critical Implementation Notes

1. **On-Premises Data Gateway Required**: ANF volumes exist within customer VNets. OneLake cannot directly access them. The gateway VM must:
   - Be deployed in same VNet as ANF endpoint
   - Have network access to the bucket endpoint
   - Run the latest gateway software
2. **Data Format Considerations**:
   - Delta tables in the `/tables` folder work best for SQL queries
   - Files folder supports any format but won't auto-discover as tables
   - Parquet recommended for analytics workloads
3. **Fabric Data Agent Limitations**:
   - Maximum 5 data sources per agent
   - Only structured data (not PDF/DOCX/TXT)
   - English language only (as of Jan 2026)
   - Read-only queries (no INSERT/UPDATE/DELETE)

2.3 Repository Structure

```
anf-onelake-fabric-agents/
├── README.md
├── LICENSE
├── docs/
│   ├── architecture.md
│   ├── prerequisites.md
│   └── troubleshooting.md
```

```
├── infra/
│   ├── bicep/
│   │   ├── main.bicep
│   │   ├── modules/
│   │   │   ├── anf-account.bicep
│   │   │   ├── anf-pool.bicep
│   │   │   ├── anf-volume.bicep
│   │   │   ├── vnet.bicep
│   │   │   └── gateway-vm.bicep
│   │   └── parameters/
│   │       ├── dev.bicepparam
│   │       └── prod.bicepparam
│   └── terraform/
│       └── (alternative IaC)
├── scripts/
│   ├── setup/
│   │   ├── 01-deploy-infrastructure.sh
│   │   ├── 02-configure-object-api.sh
│   │   ├── 03-install-gateway.ps1
│   │   └── 04-create-onelake-shortcut.md (manual steps)
│   ├── data/
│   │   ├── generate-sample-data.py
│   │   └── upload-to-anf.py
│   └── validation/
│       └── test-connectivity.py
├── fabric/
│   ├── notebooks/
│   │   ├── 01-explore-anf-data.ipynb
│   │   ├── 02-create-delta-tables.ipynb
│   │   └── 03-test-data-agent.ipynb
│   └── data-agent/
│       ├── agent-config.json
│       ├── instructions.md
│       └── example-queries.json
├── sample-data/
│   ├── transactions.parquet
│   ├── customers.csv
│   └── products.csv
└── diagrams/
    ├── architecture.drawio
    ├── architecture.png
    └── data-flow.png
```

2.4 Key Code Artifacts

Infrastructure as Code (Bicep)

```bicep
```

```
// main.bicep
targetScope = 'resourceGroup'

param location string = resourceGroup().location
param anfAccountName string
param capacityPoolName string
param volumeName string
param vnetName string
param subnetName string
param delegatedSubnetName string

// Virtual Network with delegated subnet for ANF
module vnet 'modules/vnet.bicep' = {
  name: 'vnet-deployment'
  params: {
    vnetName: vnetName
    location: location
    subnetName: subnetName
    delegatedSubnetName: delegatedSubnetName
  }
}

// Azure NetApp Files Account
module anfAccount 'modules/anf-account.bicep' = {
  name: 'anf-account-deployment'
  params: {
    accountName: anfAccountName
    location: location
  }
}

// Capacity Pool (Premium tier recommended for AI workloads)
module capacityPool 'modules/anf-pool.bicep' = {
  name: 'anf-pool-deployment'
  params: {
    accountName: anfAccount.outputs.accountName
    poolName: capacityPoolName
    location: location
    serviceLevel: 'Premium'
    size: 4398046511104 // 4 TiB minimum
  }
}
```

```
// Volume with Object REST API bucket
module volume 'modules/anf-volume.bicep' = {
  name: 'anf-volume-deployment'
  params: {
    accountName: anfAccount.outputs.accountName
    poolName: capacityPool.outputs.poolName
    volumeName: volumeName
    location: location
    subnetId: vnet.outputs.delegatedSubnetId
    protocolTypes: ['NFSv3']  // or NFSv4.1, SMB3
    quotaInBytes: 107374182400 // 100 GiB
  }
}

// Gateway VM for OneLake connectivity
module gatewayVm 'modules/gateway-vm.bicep' = {
  name: 'gateway-vm-deployment'
  params: {
    vmName: 'anf-gateway-vm'
    location: location
    subnetId: vnet.outputs.subnetId
    vmSize: 'Standard_D2s_v3'
  }
}
```

Sample Data Generator

```
python
```

```python
# scripts/data/generate-sample-data.py
"""
Generate sample sales data for the accelerator demo.
Creates Parquet files with realistic enterprise data.
"""

import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import pyarrow as pa
import pyarrow.parquet as pq
import os

def generate_transactions(num_records: int = 100000) -> pd.DataFrame:
    """Generate realistic transaction data."""
    np.random.seed(42)

    start_date = datetime(2024, 1, 1)
    end_date = datetime(2025, 12, 31)

    transactions = pd.DataFrame({
        'transaction_id': [f'TXN-{i:08d}' for i in range(num_records)],
        'customer_id': np.random.randint(1, 10001, num_records),
        'product_id': np.random.randint(1, 501, num_records),
        'quantity': np.random.randint(1, 20, num_records),
        'unit_price': np.round(np.random.uniform(10, 500, num_records), 2),
        'transaction_date': [
            start_date + timedelta(days=np.random.randint(0, 730))
            for _ in range(num_records)
        ],
        'store_region': np.random.choice(
            ['North', 'South', 'East', 'West'], num_records
        )
    })

    transactions['total_amount'] = (
        transactions['quantity'] * transactions['unit_price']
    )

    return transactions

def generate_customers(num_customers: int = 10000) -> pd.DataFrame:
    """Generate customer master data."""
```

```
    np.random.seed(42)

    first_names = ['James', 'Mary', 'John', 'Patricia', 'Robert', 'Jennifer']
    last_names = ['Smith', 'Johnson', 'Williams', 'Brown', 'Jones', 'Davis']

    customers = pd.DataFrame({
        'customer_id': range(1, num_customers + 1),
        'first_name': np.random.choice(first_names, num_customers),
        'last_name': np.random.choice(last_names, num_customers),
        'email': [f'customer{i}@example.com' for i in range(1, num_customers + 1)],
        'segment': np.random.choice(
            ['Enterprise', 'SMB', 'Consumer'],
            num_customers,
            p=[0.1, 0.3, 0.6]
        ),
        'region': np.random.choice(['North', 'South', 'East', 'West'], num_customers),
        'signup_date': [
            datetime(2020, 1, 1) + timedelta(days=np.random.randint(0, 1825))
            for _ in range(num_customers)
        ]
    })

    return customers

def generate_products(num_products: int = 500) -> pd.DataFrame:
    """Generate product catalog."""
    np.random.seed(42)

    categories = ['Electronics', 'Clothing', 'Home', 'Sports', 'Books']

    products = pd.DataFrame({
        'product_id': range(1, num_products + 1),
        'product_name': [f'Product-{i:04d}' for i in range(1, num_products + 1)],
        'category': np.random.choice(categories, num_products),
        'base_price': np.round(np.random.uniform(10, 500, num_products), 2),
        'cost': np.round(np.random.uniform(5, 250, num_products), 2),
        'supplier_id': np.random.randint(1, 51, num_products)
    })

    return products

def main():
    output_dir = 'sample-data'
    os.makedirs(output_dir, exist_ok=True)
```

```
    print("Generating transactions...")
    transactions = generate_transactions()
    pq.write_table(
        pa.Table.from_pandas(transactions),
        f'{output_dir}/transactions.parquet'
    )
    print(f"  Created {len(transactions)} transaction records")

    print("Generating customers...")
    customers = generate_customers()
    customers.to_csv(f'{output_dir}/customers.csv', index=False)
    print(f"  Created {len(customers)} customer records")

    print("Generating products...")
    products = generate_products()
    products.to_csv(f'{output_dir}/products.csv', index=False)
    print(f"  Created {len(products)} product records")

    print(f"\nSample data generated in '{output_dir}/' directory")

if __name__ == '__main__':
    main()
```

Fabric Data Agent Configuration

```
json
```

```
// fabric/data-agent/agent-config.json
{
  "name": "ANF-Sales-Analytics-Agent",
  "description": "Data Agent for querying sales analytics data stored in Azure NetApp Files via OneLake shortcut",
  "dataSources": [
    {
      "type": "lakehouse",
      "name": "sales_lakehouse",
      "tables": ["transactions", "customers", "products"]
    }
  ],
  "instructions": "This agent answers questions about sales transactions, customer data, and product information. The data con
  "exampleQueries": [
    {
      "question": "What were the top 10 products by revenue last quarter?",
      "sql": "SELECT p.product_name, p.category, SUM(t.total_amount) as revenue FROM transactions t JOIN products p ON
    },
    {
      "question": "Show customer segments with their total spending",
      "sql": "SELECT c.segment, COUNT(DISTINCT c.customer_id) as customer_count, SUM(t.total_amount) as total_spend
    },
    {
      "question": "What is the average transaction value by region?",
      "sql": "SELECT store_region, AVG(total_amount) as avg_transaction_value, COUNT(*) as transaction_count FROM tra
    }
  ]
}
```

Part 3: Scenario 2 - ANF + Azure AI Search + RAG

3.1 Architecture Overview

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                          SCENARIO 2: RAG Pipeline                           │
├─────────────────────────────────────────────────────────────────────────────┤
│                                                                             │
│  ┌──────────────────────┐                                                   │
│  │ Document Store       │  Enterprise documents (PDF, DOCX, MD)             │
│  │ (NFS/SMB)            │  continuously updated by business users           │
│  └──────────────────────┘                                                   │
│                                                                             │
```

```
│          ▼                                   │
│  ┌──────────────────────────────────────────┐                    │
│  │   Azure NetApp Files Volume      │                    │
│  │                                  │                    │
│  │  /enterprise-docs/               │                    │
│  │  ├── policies/                   │                    │
│  │  │   ├── hr-policy.pdf           │                    │
│  │  │   └── security-policy.docx    │                    │
│  │  ├── manuals/                    │                    │
│  │  │   └── product-manual.pdf      │                    │
│  │  └── faqs/                       │                    │
│  │      └── customer-faq.md         │                    │
│  │                                  │                    │
│  │  Object REST API Bucket:         │                    │
│  │  s3://enterprise-docs-bucket     │                    │
│  └──────────────────┬───────────────┘                    │
│                     │ S3-compatible access               │
│                     │                                    │
│   ┌─────────────────┴─────────────┐                      │
│   │                               │                      │
│   ▼                               ▼                      │
│ ┌──────────────┐  ┌──────────────────────────────────┐  │
│ │ OneLake      │  │   Azure AI Search                │  │
│ │ Shortcut     │  │                                  │  │
│ │ (metadata)   │  │  Indexer Pipeline:               │  │
│ └──────────────┘  │  1. Pull docs from Blob (copied) │  │
│                   │  2. Crack documents              │  │
│                   │  3. Chunk content                │  │
│                   │  4. Generate embeddings (ADA-002) │  │
│                   │  5. Store vectors + metadata     │  │
│                   │                                  │  │
│                   │  Index: enterprise-docs-index    │  │
│                   │  - chunk_id (key)                │  │
│                   │  - parent_id                     │  │
│                   │  - chunk (text)                  │  │
│                   │  - chunk_vector (1536 dims)      │  │
│                   │  - title, path, category         │  │
│                   └──────────────┬───────────────────┘  │
│                                  │ Vector + Hybrid Search │
│                                  ▼                      │
│                   ┌──────────────────────────────────┐  │
│                   │   Azure OpenAI                   │  │
│                   │                                  │  │
│                   │  RAG Chat Application:           │  │
```

```
│                │ 1. Receive user question          │              │
│                │ 2. Embed question (ADA-002)       │              │
│                │ 3. Hybrid search (vector + keyword) │            │
│                │ 4. Retrieve top-k chunks          │              │
│                │ 5. Generate answer (GPT-4o)       │              │
│                │ 6. Return with citations          │              │
│                └──────────────────────────────────────────────────┘
│                                                                   │
│                                                                   │
│
│
│  ┌──────────────────────────────────────────────────
```

- Azure SQL Database
- Azure Cosmos DB
- Azure Files (SMB shares, preview)

Recommended Pattern: Sync ANF → Blob → AI Search

```
ANF Volume ──(scheduled sync)──▶ Blob Storage ──(indexer)──▶ AI Search Index
    │                                  │
    │                                  │
    └──────────── NFS/SMB access for document management ────────────┘
          (primary source of truth)
```

Sync Implementation Options:

1. **Azure Data Factory Pipeline** (Recommended)
    - Scheduled copy activity
    - Supports incremental copy via watermarks
    - Enterprise-grade monitoring
2. **Azure Functions + File Access Logging**
    - Near real-time sync
    - Triggered by ANF file access logs
    - More complex implementation
3. **Custom Python Script**
    - Simple scheduled sync via cron/Azure Automation
    - Uses Object REST API to list/read
    - Uses Blob SDK to write

3.3 Repository Structure

```
anf-ai-search-rag/
├── README.md
├── LICENSE
├── docs/
│   ├── architecture.md
│   ├── data-sync-patterns.md
│   └── rag-tuning-guide.md
├── infra/
│   ├── bicep/
```

```
│   │   ├── main.bicep
│   │   ├── modules/
│   │   │   ├── anf-volume.bicep
│   │   │   ├── storage-account.bicep
│   │   │   ├── ai-search.bicep
│   │   │   ├── openai.bicep
│   │   │   └── data-factory.bicep
│   │   └── parameters/
│   └── terraform/
├── sync/
│   ├── data-factory/
│   │   ├── pipeline-definition.json
│   │   └── linked-services/
│   ├── python-sync/
│   │   ├── anf_to_blob_sync.py
│   │   ├── requirements.txt
│   │   └── Dockerfile
│   └── function-trigger/
│       └── (event-driven option)
├── search/
│   ├── index/
│   │   ├── index-schema.json
│   │   ├── skillset.json
│   │   ├── data-source.json
│   │   └── indexer.json
│   └── scripts/
│       ├── create-index.py
│       ├── create-skillset.py
│       └── run-indexer.py
├── app/
│   ├── backend/
│   │   ├── main.py
│   │   ├── rag_service.py
│   │   ├── search_client.py
│   │   ├── requirements.txt
│   │   └── Dockerfile
│   └── frontend/
│       ├── (React/Streamlit chat UI)
│       └── ...
├── notebooks/
│   ├── 01-setup-and-test.ipynb
│   ├── 02-search-testing.ipynb
│   └── 03-rag-evaluation.ipynb
├── sample-docs/
```

```
│   ├── policies/
│   ├── manuals/
│   └── faqs/
└── diagrams/
```

3.4 Key Code Artifacts

Azure AI Search Index Schema

```
json
```

```
// search/index/index-schema.json
{
  "name": "enterprise-docs-index",
  "fields": [
    {
      "name": "chunk_id",
      "type": "Edm.String",
      "key": true,
      "filterable": true,
      "sortable": true
    },
    {
      "name": "parent_id",
      "type": "Edm.String",
      "filterable": true
    },
    {
      "name": "chunk",
      "type": "Edm.String",
      "searchable": true,
      "analyzer": "en.microsoft"
    },
    {
      "name": "chunk_vector",
      "type": "Collection(Edm.Single)",
      "searchable": true,
      "vectorSearchDimensions": 1536,
      "vectorSearchProfileName": "vector-profile"
    },
    {
      "name": "title",
      "type": "Edm.String",
      "searchable": true,
      "filterable": true
    },
    {
      "name": "file_path",
      "type": "Edm.String",
      "filterable": true
    },
    {
      "name": "category",
      "type": "Edm.String",
```

```
    "filterable": true,
    "facetable": true
  },
  {
    "name": "last_modified",
    "type": "Edm.DateTimeOffset",
    "filterable": true,
    "sortable": true
  }
],
"vectorSearch": {
  "algorithms": [
    {
      "name": "hnsw-algorithm",
      "kind": "hnsw",
      "hnswParameters": {
        "metric": "cosine",
        "m": 4,
        "efConstruction": 400,
        "efSearch": 500
      }
    }
  ],
  "profiles": [
    {
      "name": "vector-profile",
      "algorithm": "hnsw-algorithm",
      "vectorizer": "openai-vectorizer"
    }
  ],
  "vectorizers": [
    {
      "name": "openai-vectorizer",
      "kind": "azureOpenAI",
      "azureOpenAIParameters": {
        "resourceUri": "${AZURE_OPENAI_ENDPOINT}",
        "deploymentId": "text-embedding-3-large",
        "modelName": "text-embedding-3-large"
      }
    }
  ]
},
"semantic": {
  "configurations": [
```

```
    {
      "name": "semantic-config",
      "prioritizedFields": {
        "contentFields": [{ "fieldName": "chunk" }],
        "titleField": { "fieldName": "title" }
      }
    }
  ]
 }
}
```

RAG Service Implementation

```python
```

```python
# app/backend/rag_service.py
"""
RAG Service for enterprise document Q&A.
Implements hybrid search with Azure AI Search and answer generation with Azure OpenAI.
"""

from azure.search.documents import SearchClient
from azure.search.documents.models import VectorizedQuery
from azure.core.credentials import AzureKeyCredential
from openai import AzureOpenAI
import os
from typing import List, Dict, Any

class RAGService:
    def __init__(self):
        # Azure AI Search client
        self.search_client = SearchClient(
            endpoint=os.environ["AZURE_SEARCH_ENDPOINT"],
            index_name=os.environ["AZURE_SEARCH_INDEX"],
            credential=AzureKeyCredential(os.environ["AZURE_SEARCH_KEY"])
        )

        # Azure OpenAI client
        self.openai_client = AzureOpenAI(
            azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
            api_key=os.environ["AZURE_OPENAI_KEY"],
            api_version="2024-10-21"
        )

        self.embedding_model = "text-embedding-3-large"
        self.chat_model = "gpt-4o"

    def get_embedding(self, text: str) -> List[float]:
        """Generate embedding for query text."""
        response = self.openai_client.embeddings.create(
            input=text,
            model=self.embedding_model
        )
        return response.data[0].embedding

    def hybrid_search(
        self,
        query: str,
```

```
    top_k: int = 5,
    category_filter: str = None
) -> List[Dict[str, Any]]:
    """
    Perform hybrid search (vector + keyword) with optional filtering.
    """
    # Generate query embedding
    query_vector = self.get_embedding(query)

    # Build vector query
    vector_query = VectorizedQuery(
        vector=query_vector,
        k_nearest_neighbors=top_k,
        fields="chunk_vector"
    )

    # Build filter
    filter_expr = None
    if category_filter:
        filter_expr = f"category eq '{category_filter}'"

    # Execute hybrid search
    results = self.search_client.search(
        search_text=query,  # Keyword component
        vector_queries=[vector_query],  # Vector component
        filter=filter_expr,
        select=["chunk_id", "chunk", "title", "file_path", "category"],
        top=top_k,
        query_type="semantic",
        semantic_configuration_name="semantic-config"
    )

    return [
        {
            "chunk_id": r["chunk_id"],
            "content": r["chunk"],
            "title": r["title"],
            "path": r["file_path"],
            "category": r["category"],
            "score": r["@search.score"]
        }
        for r in results
    ]
```

```
    def generate_answer(
        self,
        query: str,
        context_chunks: List[Dict[str, Any]]
    ) -> Dict[str, Any]:
        """
        Generate answer using retrieved context.
        """
        # Format context
        context_text = "\n\n".join([
            f"[Source: {c['title']} ({c['path']})]\n{c['content']}"
            for c in context_chunks
        ])

        # System prompt
        system_prompt = """You are an enterprise document assistant.
Answer questions based ONLY on the provided context.
If the answer is not in the context, say "I don't have information about that in the available documents."
Always cite your sources using [Source: filename] format."""

        # Generate response
        response = self.openai_client.chat.completions.create(
            model=self.chat_model,
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": f"Context:\n{context_text}\n\nQuestion: {query}"}
            ],
            temperature=0.3,
            max_tokens=1000
        )

        return {
            "answer": response.choices[0].message.content,
            "sources": [
                {"title": c["title"], "path": c["path"]}
                for c in context_chunks
            ],
            "model": self.chat_model
        }

    def ask(
        self,
        query: str,
        top_k: int = 5,
```

```python
    category_filter: str = None
) -> Dict[str, Any]:
    """
    Complete RAG pipeline: search + generate.
    """
    # Retrieve relevant chunks
    chunks = self.hybrid_search(query, top_k, category_filter)

    if not chunks:
        return {
            "answer": "I couldn't find any relevant documents to answer your question.",
            "sources": [],
            "chunks_retrieved": 0
        }

    # Generate answer
    result = self.generate_answer(query, chunks)
    result["chunks_retrieved"] = len(chunks)

    return result
```

Data Sync Script (ANF to Blob)

```python
```

```python
# sync/python-sync/anf_to_blob_sync.py
"""
Sync documents from Azure NetApp Files (via Object REST API) to Azure Blob Storage.
This enables Azure AI Search indexer to process the documents.
"""

import boto3
from azure.storage.blob import BlobServiceClient
from azure.identity import DefaultAzureCredential
import os
import logging
from datetime import datetime
from typing import Set

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class ANFBlobSync:
    def __init__(self):
        # ANF Object REST API configuration
        self.anf_endpoint = os.environ["ANF_BUCKET_ENDPOINT"]
        self.anf_bucket = os.environ["ANF_BUCKET_NAME"]
        self.anf_access_key = os.environ["ANF_ACCESS_KEY"]
        self.anf_secret_key = os.environ["ANF_SECRET_KEY"]

        # Azure Blob configuration
        self.blob_connection_string = os.environ["BLOB_CONNECTION_STRING"]
        self.blob_container = os.environ["BLOB_CONTAINER_NAME"]

        # Initialize clients
        self.s3_client = boto3.client(
            's3',
            endpoint_url=self.anf_endpoint,
            aws_access_key_id=self.anf_access_key,
            aws_secret_access_key=self.anf_secret_key,
            region_name='us-east-1',  # Required for ANF
            verify=False  # Adjust based on certificate setup
        )

        self.blob_service = BlobServiceClient.from_connection_string(
            self.blob_connection_string
        )
        self.container_client = self.blob_service.get_container_client(
```

```
        self.blob_container
    )

def list_anf_objects(self, prefix: str = "") -> Set[str]:
    """List all objects in ANF bucket."""
    objects = set()
    paginator = self.s3_client.get_paginator('list_objects_v2')

    for page in paginator.paginate(Bucket=self.anf_bucket, Prefix=prefix):
        for obj in page.get('Contents', []):
            objects.add(obj['Key'])

    return objects

def list_blob_objects(self) -> Set[str]:
    """List all blobs in container."""
    return set(
        blob.name
        for blob in self.container_client.list_blobs()
    )

def sync_object(self, key: str) -> bool:
    """Copy single object from ANF to Blob."""
    try:
        # Download from ANF
        response = self.s3_client.get_object(
            Bucket=self.anf_bucket,
            Key=key
        )
        content = response['Body'].read()

        # Upload to Blob
        blob_client = self.container_client.get_blob_client(key)
        blob_client.upload_blob(content, overwrite=True)

        logger.info(f"Synced: {key}")
        return True

    except Exception as e:
        logger.error(f"Failed to sync {key}: {e}")
        return False

def delete_blob(self, key: str) -> bool:
    """Delete blob that no longer exists in source."""
```

```
        try:
            blob_client = self.container_client.get_blob_client(key)
            blob_client.delete_blob()
            logger.info(f"Deleted: {key}")
            return True
        except Exception as e:
            logger.error(f"Failed to delete {key}: {e}")
            return False

    def run_sync(self, prefix: str = "", delete_orphans: bool = True):
        """
        Full sync: copy new/updated files, optionally delete orphans.
        """
        logger.info(f"Starting sync at {datetime.utcnow().isoformat()}")

        # Get current state
        anf_objects = self.list_anf_objects(prefix)
        blob_objects = self.list_blob_objects()

        # Files to copy (new or potentially updated)
        to_copy = anf_objects  # Simplified: copy all each run
        # For production: compare ETags/timestamps for incremental sync

        # Files to delete (in blob but not in ANF)
        to_delete = blob_objects - anf_objects if delete_orphans else set()

        # Execute sync
        copied = sum(1 for key in to_copy if self.sync_object(key))
        deleted = sum(1 for key in to_delete if self.delete_blob(key))

        logger.info(f"Sync complete: {copied} copied, {deleted} deleted")

        return {
            "copied": copied,
            "deleted": deleted,
            "timestamp": datetime.utcnow().isoformat()
        }

if __name__ == "__main__":
    syncer = ANFBlobSync()
    syncer.run_sync()
```

Part 4: Scenario 3 - Foundry Agents + ANF SelfOps

4.1 Architecture Overview

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                    SCENARIO 3: Agentic SelfOps                              │
├─────────────────────────────────────────────────────────────────────────────┤
│                                                                             │
│  ┌──────────────────────────────────────────────┐                           │
│  │   Azure AI Foundry Agent                     │                           │
│  │                                              │                           │
│  │  Agent: "ANF-DataOps-Agent"                  │                           │
│  │  Model: gpt-4o                               │                           │
│  │                                              │                           │
│  │  Instructions:                               │                           │
│  │  "You are a data operations agent            │                           │
│  │   that manages Azure NetApp Files            │                           │
│  │   storage. You can create snapshots,         │                           │
│  │   list volumes, and manage data              │                           │
│  │   protection for AI workloads."              │                           │
│  │                                              │                           │
│  │  Tools:                                      │                           │
│  │  ├── create_snapshot()                       │                           │
│  │  ├── list_snapshots()                        │                           │
│  │  ├── restore_snapshot()                      │                           │
│  │  ├── list_volumes()                          │                           │
│  │  ├── get_volume_metrics()                    │                           │
│  │  └── resize_volume()                         │                           │
│  └──────────────────────┬──────────────────────┘                           │
│                         │ Function Calling                                 │
│                         │ (requires_action state)                          │
│                         ▼                                                  │
│  ┌──────────────────────────────────────────────┐                           │
│  │   Tool Execution Layer                       │                           │
│  │   (Your Application Code)                    │                           │
│  │                                              │                           │
│  │  def execute_tool(tool_call):                │                           │
│  │      if tool_call.name == "create_snapshot":                             │
│  │          return anf_client.create_snapshot(...)                          │
│  │      elif tool_call.name == "list_volumes":                              │
│  │          return anf_client.list_volumes(...)                             │
│  │      ...                                     │                           │
│  └──────────────────────┬──────────────────────┘                           │
```

```
│              │ azure-mgmt-netapp SDK                          │
│              ▼                                                │
│  ┌──────────────────────────────────────────────────────────┐
│  │   Azure NetApp Files                  │                  │
│  │   Control Plane REST API              │                  │
│  │                                       │                  │
│  │ Endpoints:                            │                  │
│  │ PUT  .../snapshots/{name}             │ Create snapshot  │
│  │ GET  .../snapshots                    │ List snapshots   │
│  │ POST .../revert                       │ Restore snapshot │
│  │ GET  .../volumes                      │ List volumes     │
│  │ GET  .../volumes/{name}               │ Get volume details │
│  │ PATCH .../volumes/{name}              │ Resize/modify volume │
│  └──────────────────────────────────────────────────────────┘
│                                                               │
└───────────────────────────────────────────────────────────────┘

┌───────────────────────────────────────────────────────────────┐
│                                                               │
│  │                EXAMPLE WORKFLOW                          │ │
│  │                                                          │ │
│  │ User: "Create a pre-training snapshot of the ML volume"  │ │
│  │                                                          │ │
│  │ Agent: Calls create_snapshot(volume="ml-data", name="pre-training") │ │
│  │       ↓                                                  │ │
│  │ Tool Layer: Executes azure-mgmt-netapp API call          │ │
│  │       ↓                                                  │ │
│  │ ANF: Creates snapshot (< 30 seconds)                     │ │
│  │       ↓                                                  │ │
│  │ Agent: "Created snapshot 'pre-training' on volume 'ml-data'. │ │
│  │        Snapshot ID: /subscriptions/.../snapshots/pre-training" │ │
│                                                               │
└───────────────────────────────────────────────────────────────┘

│                                                               │
│                                                               │
└───────────────────────────────────────────────────────────────┘
```

4.2 Technical Viability Assessment

Component	Status	Notes
Azure AI Foundry Agent Service	✅ GA (Jan 2025)	Function calling supported
azure-ai-projects SDK	✅ GA	Python/C#/.NET/JS

Component	Status	Notes
azure-mgmt-netapp SDK	✅ GA	v14.0.1, API 2024-07-01
Function Calling	✅ GA	Custom tool definitions

Security Considerations

1. **Service Principal Required**: The tool execution layer needs an Azure AD service principal with:
   - `Microsoft.NetApp/netAppAccounts/capacityPools/volumes/snapshots/*` permissions
   - Scoped to specific resource group/subscription
2. **RBAC Best Practices**:
   - Create custom role with least privilege
   - Use Managed Identity where possible
   - Implement approval workflow for destructive operations
3. **Guardrails**:
   - Validate snapshot names (no special characters)
   - Enforce naming conventions
   - Limit restore operations to specific volumes
   - Log all agent actions for audit

4.3 Repository Structure

```
anf-foundry-selfops/
├── README.md
├── LICENSE
├── docs/
│   ├── architecture.md
│   ├── security-guide.md
│   └── tool-reference.md
├── infra/
│   ├── bicep/
│   │   ├── main.bicep
│   │   ├── modules/
│   │   │   ├── anf-resources.bicep
│   │   │   ├── ai-foundry.bicep
│   │   │   └── rbac-assignments.bicep
│   │   └── custom-roles/
│   │       └── anf-agent-operator.json
```

```
│   └── terraform/
├── agent/
│   ├── tools/
│   │   ├── __init__.py
│   │   ├── snapshot_tools.py
│   │   ├── volume_tools.py
│   │   └── metrics_tools.py
│   ├── definitions/
│   │   └── tool_definitions.json
│   ├── agent_config.py
│   └── main.py
├── src/
│   ├── anf_client/
│   │   ├── __init__.py
│   │   ├── client.py
│   │   ├── snapshot_operations.py
│   │   ├── volume_operations.py
│   │   └── models.py
│   └── utils/
│       ├── auth.py
│       └── validation.py
├── tests/
│   ├── unit/
│   └── integration/
├── notebooks/
│   ├── 01-setup-agent.ipynb
│   ├── 02-test-tools.ipynb
│   └── 03-demo-scenarios.ipynb
├── requirements.txt
└── diagrams/
```

4.4 Key Code Artifacts

Tool Definitions

```
json
```

```
// agent/definitions/tool_definitions.json
{
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "create_snapshot",
        "description": "Create a point-in-time snapshot of an Azure NetApp Files volume. Use this before any risky operation lik
        "parameters": {
          "type": "object",
          "properties": {
            "volume_name": {
              "type": "string",
              "description": "Name of the ANF volume to snapshot"
            },
            "snapshot_name": {
              "type": "string",
              "description": "Name for the new snapshot (alphanumeric and hyphens only, max 64 chars)"
            },
            "description": {
              "type": "string",
              "description": "Optional description of why this snapshot is being created"
            }
          },
          "required": ["volume_name", "snapshot_name"]
        }
      }
    },
    {
      "type": "function",
      "function": {
        "name": "list_snapshots",
        "description": "List all snapshots for a specified Azure NetApp Files volume. Returns snapshot names, creation times, an
        "parameters": {
          "type": "object",
          "properties": {
            "volume_name": {
              "type": "string",
              "description": "Name of the ANF volume"
            }
          },
          "required": ["volume_name"]
        }
```

```
    }
  },
  {
    "type": "function",
    "function": {
      "name": "restore_snapshot",
      "description": "Restore a volume to a previous snapshot state. WARNING: This is a destructive operation that will revert
      "parameters": {
        "type": "object",
        "properties": {
          "volume_name": {
            "type": "string",
            "description": "Name of the ANF volume to restore"
          },
          "snapshot_name": {
            "type": "string",
            "description": "Name of the snapshot to restore to"
          },
          "confirmation": {
            "type": "string",
            "description": "Must be 'CONFIRM_RESTORE' to proceed with this destructive operation"
          }
        },
        "required": ["volume_name", "snapshot_name", "confirmation"]
      }
    }
  },
  {
    "type": "function",
    "function": {
      "name": "list_volumes",
      "description": "List all Azure NetApp Files volumes in the configured scope. Returns volume names, sizes, protocols, an
      "parameters": {
        "type": "object",
        "properties": {
          "capacity_pool": {
            "type": "string",
            "description": "Optional: filter to a specific capacity pool"
          }
        }
      }
    }
  },
  {
```

```
  "type": "function",
  "function": {
    "name": "get_volume_metrics",
    "description": "Get current metrics for an ANF volume including used capacity, throughput, IOPS, and latency.",
    "parameters": {
      "type": "object",
      "properties": {
        "volume_name": {
          "type": "string",
          "description": "Name of the ANF volume"
        },
        "time_range": {
          "type": "string",
          "enum": ["1h", "6h", "24h", "7d"],
          "description": "Time range for metrics aggregation"
        }
      },
      "required": ["volume_name"]
    }
  }
},
{
  "type": "function",
  "function": {
    "name": "resize_volume",
    "description": "Resize an ANF volume capacity. This is an online operation with no downtime. Volumes can be increase
    "parameters": {
      "type": "object",
      "properties": {
        "volume_name": {
          "type": "string",
          "description": "Name of the ANF volume"
        },
        "new_size_gib": {
          "type": "integer",
          "description": "New size in GiB (minimum 100 GiB)"
        }
      },
      "required": ["volume_name", "new_size_gib"]
    }
  }
}
```

```
  ]
}
```

ANF Client Implementation

```python
```

```python
# src/anf_client/client.py
"""
Azure NetApp Files client for agent tool operations.
Wraps azure-mgmt-netapp SDK with simplified interface.
"""

from azure.identity import DefaultAzureCredential, ClientSecretCredential
from azure.mgmt.netapp import NetAppManagementClient
from azure.mgmt.netapp.models import Snapshot
from typing import List, Dict, Any, Optional
import os
import re
from datetime import datetime

class ANFClient:
    """Client for Azure NetApp Files management operations."""

    def __init__(
        self,
        subscription_id: str = None,
        resource_group: str = None,
        account_name: str = None,
        pool_name: str = None
    ):
        """
        Initialize ANF client with configuration from environment or parameters.
        """
        self.subscription_id = subscription_id or os.environ["AZURE_SUBSCRIPTION_ID"]
        self.resource_group = resource_group or os.environ["ANF_RESOURCE_GROUP"]
        self.account_name = account_name or os.environ["ANF_ACCOUNT_NAME"]
        self.pool_name = pool_name or os.environ["ANF_POOL_NAME"]

        # Initialize credential and client
        self.credential = DefaultAzureCredential()
        self.client = NetAppManagementClient(
            self.credential,
            self.subscription_id
        )

    def _validate_name(self, name: str, resource_type: str) -> bool:
        """Validate resource name format."""
        # ANF naming: alphanumeric and hyphens, 1-64 characters
        pattern = r'^[a-zA-Z0-9][-a-zA-Z0-9]{0,62}[a-zA-Z0-9]?$'
```

```
    if not re.match(pattern, name):
        raise ValueError(
            f"Invalid {resource_type} name '{name}'. "
            f"Must be 1-64 alphanumeric characters and hyphens."
        )
    return True

# ========== Snapshot Operations ==========

def create_snapshot(
    self,
    volume_name: str,
    snapshot_name: str,
    description: str = None
) -> Dict[str, Any]:
    """
    Create a snapshot of the specified volume.

    Returns:
        Dict with snapshot_id, name, created_at, and status
    """
    self._validate_name(snapshot_name, "snapshot")

    # Get volume location
    volume = self.client.volumes.get(
        self.resource_group,
        self.account_name,
        self.pool_name,
        volume_name
    )

    # Create snapshot (async operation)
    poller = self.client.snapshots.begin_create(
        resource_group_name=self.resource_group,
        account_name=self.account_name,
        pool_name=self.pool_name,
        volume_name=volume_name,
        snapshot_name=snapshot_name,
        body=Snapshot(location=volume.location)
    )

    result = poller.result()

    return {
```

```
        "snapshot_id": result.id,
        "name": result.name.split("/")[-1],
        "created_at": result.created.isoformat() if result.created else None,
        "provisioning_state": result.provisioning_state,
        "volume": volume_name,
        "description": description
    }

def list_snapshots(self, volume_name: str) -> List[Dict[str, Any]]:
    """List all snapshots for a volume."""
    snapshots = self.client.snapshots.list(
        self.resource_group,
        self.account_name,
        self.pool_name,
        volume_name
    )

    return [
        {
            "name": s.name.split("/")[-1],
            "snapshot_id": s.id,
            "created_at": s.created.isoformat() if s.created else None,
            "provisioning_state": s.provisioning_state
        }
        for s in snapshots
    ]

def delete_snapshot(
    self,
    volume_name: str,
    snapshot_name: str
) -> Dict[str, Any]:
    """Delete a snapshot."""
    poller = self.client.snapshots.begin_delete(
        self.resource_group,
        self.account_name,
        self.pool_name,
        volume_name,
        snapshot_name
    )

    poller.result()  # Wait for completion

    return {
```

```
        "status": "deleted",
        "snapshot_name": snapshot_name,
        "volume": volume_name
    }

def restore_snapshot(
    self,
    volume_name: str,
    snapshot_name: str,
    confirmation: str
) -> Dict[str, Any]:
    """
    Restore volume to snapshot state.
    Requires explicit confirmation due to destructive nature.
    """
    if confirmation != "CONFIRM_RESTORE":
        raise ValueError(
            "Restore operation requires confirmation='CONFIRM_RESTORE'"
        )

    # Get snapshot ID
    snapshot = self.client.snapshots.get(
        self.resource_group,
        self.account_name,
        self.pool_name,
        volume_name,
        snapshot_name
    )

    # Initiate restore (revert)
    poller = self.client.volumes.begin_revert(
        self.resource_group,
        self.account_name,
        self.pool_name,
        volume_name,
        body={"snapshot_id": snapshot.snapshot_id}
    )

    poller.result()

    return {
        "status": "restored",
        "volume": volume_name,
        "restored_to_snapshot": snapshot_name,
```

```
        "restored_at": datetime.utcnow().isoformat()
    }

# ========== Volume Operations ==========

def list_volumes(
    self,
    capacity_pool: str = None
) -> List[Dict[str, Any]]:
    """List all volumes, optionally filtered by pool."""
    pool = capacity_pool or self.pool_name

    volumes = self.client.volumes.list(
        self.resource_group,
        self.account_name,
        pool
    )

    return [
        {
            "name": v.name.split("/")[-1],
            "volume_id": v.id,
            "location": v.location,
            "quota_gib": v.usage_threshold / (1024**3),
            "protocol_types": v.protocol_types,
            "service_level": v.service_level,
            "throughput_mibps": v.throughput_mibps,
            "provisioning_state": v.provisioning_state
        }
        for v in volumes
    ]

def get_volume(self, volume_name: str) -> Dict[str, Any]:
    """Get details for a specific volume."""
    v = self.client.volumes.get(
        self.resource_group,
        self.account_name,
        self.pool_name,
        volume_name
    )

    return {
        "name": v.name.split("/")[-1],
        "volume_id": v.id,
```

```
        "location": v.location,
        "quota_gib": v.usage_threshold / (1024**3),
        "protocol_types": v.protocol_types,
        "service_level": v.service_level,
        "throughput_mibps": v.throughput_mibps,
        "provisioning_state": v.provisioning_state,
        "subnet_id": v.subnet_id,
        "creation_token": v.creation_token,
        "export_policy": str(v.export_policy) if v.export_policy else None
    }

def resize_volume(
    self,
    volume_name: str,
    new_size_gib: int
) -> Dict[str, Any]:
    """
    Resize volume to new quota.
    Online operation, no downtime required.
    """
    if new_size_gib < 100:
        raise ValueError("Minimum volume size is 100 GiB")

    new_size_bytes = new_size_gib * (1024**3)

    poller = self.client.volumes.begin_update(
        self.resource_group,
        self.account_name,
        self.pool_name,
        volume_name,
        body={"usage_threshold": new_size_bytes}
    )

    result = poller.result()

    return {
        "status": "resized",
        "volume": volume_name,
        "new_size_gib": new_size_gib,
        "provisioning_state": result.provisioning_state
    }
```

Agent Main Implementation

python

```python
# agent/main.py
"""
Azure AI Foundry Agent with ANF storage management tools.
Demonstrates Agentic SelfOps pattern.
"""

import os
import json
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient
from azure.ai.agents.models import FunctionTool
from typing import Dict, Any, Callable

from src.anf_client.client import ANFClient
from agent.tools.snapshot_tools import SnapshotTools
from agent.tools.volume_tools import VolumeTools

class ANFDataOpsAgent:
    """Agent for Azure NetApp Files data operations."""

    def __init__(self):
        # Initialize AI Foundry client
        self.project_endpoint = os.environ["PROJECT_ENDPOINT"]
        self.model_deployment = os.environ.get("MODEL_DEPLOYMENT", "gpt-4o")

        self.credential = DefaultAzureCredential()
        self.project_client = AIProjectClient(
            endpoint=self.project_endpoint,
            credential=self.credential
        )

        # Initialize ANF client
        self.anf_client = ANFClient()

        # Initialize tools
        self.snapshot_tools = SnapshotTools(self.anf_client)
        self.volume_tools = VolumeTools(self.anf_client)

        # Build tool registry
        self.tool_registry: Dict[str, Callable] = {
            "create_snapshot": self.snapshot_tools.create_snapshot,
            "list_snapshots": self.snapshot_tools.list_snapshots,
            "restore_snapshot": self.snapshot_tools.restore_snapshot,
```

```
        "list_volumes": self.volume_tools.list_volumes,
        "get_volume_metrics": self.volume_tools.get_volume_metrics,
        "resize_volume": self.volume_tools.resize_volume,
    }

    # Load tool definitions
    with open("agent/definitions/tool_definitions.json") as f:
        self.tool_definitions = json.load(f)["tools"]

    # Create agent
    self.agent = None
    self._create_agent()

def _create_agent(self):
    """Create the Foundry agent with tools."""
    # Convert tool definitions to FunctionTool objects
    tools = [
        FunctionTool(
            name=t["function"]["name"],
            description=t["function"]["description"],
            parameters=t["function"]["parameters"]
        )
        for t in self.tool_definitions
    ]

    # Create agent
    self.agent = self.project_client.agents.create_agent(
        model=self.model_deployment,
        name="ANF-DataOps-Agent",
        instructions="""You are a data operations agent that manages Azure NetApp Files storage for AI workloads.

Your capabilities:
- Create point-in-time snapshots before risky operations (training, ingestion, migrations)
- List and manage existing snapshots for recovery planning
- Restore volumes to previous states when needed (requires explicit confirmation)
- Monitor volume capacity and performance metrics
- Resize volumes based on workload needs

Always:
- Explain what you're doing before taking action
- Create snapshots before any destructive or risky operation
- Require explicit confirmation for restore operations
- Report results clearly with relevant details (IDs, timestamps, sizes)
```

```
Safety guidelines:
- Never delete the last remaining snapshot of a volume
- Warn users about data loss implications of restore operations
- Validate input parameters before executing operations""",
            tools=tools
        )

        print(f"Created agent: {self.agent.id}")

    def execute_tool(self, tool_name: str, arguments: Dict[str, Any]) -> str:
        """Execute a tool and return the result as a string."""
        if tool_name not in self.tool_registry:
            return json.dumps({"error": f"Unknown tool: {tool_name}"})

        try:
            result = self.tool_registry[tool_name](**arguments)
            return json.dumps(result, default=str)
        except Exception as e:
            return json.dumps({"error": str(e)})

    def run(self, user_message: str) -> str:
        """
        Run the agent with a user message.
        Handles the full conversation including tool calls.
        """
        # Create thread
        thread = self.project_client.agents.create_thread()

        # Add user message
        self.project_client.agents.create_message(
            thread_id=thread.id,
            role="user",
            content=user_message
        )

        # Create run
        run = self.project_client.agents.create_run(
            thread_id=thread.id,
            assistant_id=self.agent.id
        )

        # Poll for completion, handling tool calls
        while True:
            run = self.project_client.agents.get_run(
```

```
            thread_id=thread.id,
            run_id=run.id
        )

        if run.status == "completed":
            break
        elif run.status == "requires_action":
            # Handle tool calls
            tool_calls = run.required_action.submit_tool_outputs.tool_calls
            tool_outputs = []

            for tc in tool_calls:
                arguments = json.loads(tc.function.arguments)
                output = self.execute_tool(tc.function.name, arguments)
                tool_outputs.append({
                    "tool_call_id": tc.id,
                    "output": output
                })

            # Submit tool outputs
            run = self.project_client.agents.submit_tool_outputs_to_run(
                thread_id=thread.id,
                run_id=run.id,
                tool_outputs=tool_outputs
            )
        elif run.status in ["failed", "cancelled", "expired"]:
            return f"Run failed with status: {run.status}"

        # Brief pause before polling again
        import time
        time.sleep(1)

    # Get final messages
    messages = self.project_client.agents.list_messages(thread_id=thread.id)

    # Return the last assistant message
    for msg in messages.data:
        if msg.role == "assistant":
            return msg.content[0].text.value

    return "No response generated"

def cleanup(self):
    """Delete the agent when done."""
```

```
        if self.agent:
            self.project_client.agents.delete_agent(self.agent.id)
            print(f"Deleted agent: {self.agent.id}")

# Example usage
if __name__ == "__main__":
    agent = ANFDataOpsAgent()

    try:
        # Demo conversation
        print("=" * 60)
        print("ANF DataOps Agent Demo")
        print("=" * 60)

        # Example 1: List volumes
        print("\n[User]: What volumes are available?")
        response = agent.run("What volumes are available in my storage account?")
        print(f"[Agent]: {response}")

        # Example 2: Create snapshot
        print("\n[User]: Create a snapshot before training")
        response = agent.run(
            "I'm about to start model training. "
            "Please create a safety snapshot of the ml-data volume."
        )
        print(f"[Agent]: {response}")

        # Example 3: List snapshots
        print("\n[User]: Show me available recovery points")
        response = agent.run("What snapshots exist for the ml-data volume?")
        print(f"[Agent]: {response}")

    finally:
        agent.cleanup()
```

Part 5: Implementation Roadmap

5.1 Prerequisites Checklist

Requirement	Scenario 1	Scenario 2	Scenario 3
Azure Subscription	✅	✅	✅

Requirement	Scenario 1	Scenario 2	Scenario 3
Azure NetApp Files Account	✅	✅	✅
Object REST API Waitlist Approval	✅	✅	❌
SSL Certificate (PEM)	✅	✅	❌
Microsoft Fabric License (F2+)	✅	❌	❌
On-Premises Data Gateway	✅	❌	❌
Azure AI Search (Standard+)	❌	✅	❌
Azure OpenAI Access	❌	✅	✅
Azure AI Foundry Project	❌	❌	✅
Service Principal (Contributor)	✅	✅	✅

5.2 Development Timeline

```
Week 1: Foundation
├── Day 1-2: Infrastructure setup (Bicep/Terraform)
├── Day 3-4: ANF configuration and Object REST API enrollment
└── Day 5: Sample data generation and upload

Week 2: Scenario 1 (OneLake + Fabric Agents)
├── Day 1: Gateway VM deployment and configuration
├── Day 2: OneLake shortcut creation and testing
├── Day 3: Lakehouse setup and data validation
├── Day 4: Fabric Data Agent creation and tuning
└── Day 5: Documentation and notebooks

Week 3: Scenario 2 (AI Search + RAG)
├── Day 1: Blob storage setup and sync mechanism
├── Day 2: Azure AI Search index and skillset
├── Day 3: Indexer configuration and testing
├── Day 4: RAG application backend
└── Day 5: Frontend and end-to-end testing

Week 4: Scenario 3 (Foundry Agents + SelfOps)
├── Day 1: ANF client library
├── Day 2: Tool implementations
├── Day 3: Foundry Agent setup
```

```
├── Day 4: Integration testing
└── Day 5: Security hardening and documentation
```

5.3 Testing Matrix

Test Type	Scenario 1	Scenario 2	Scenario 3
Unit Tests	Data generators	RAG service	ANF client
Integration Tests	OneLake connectivity	Search + OpenAI	Agent + ANF API
E2E Tests	Data Agent queries	Chat interactions	Tool execution
Performance Tests	Query latency	Search/RAG latency	Snapshot time
Security Tests	Gateway access	Auth flows	RBAC validation

Part 6: Presentation Talking Points

Key Messages for Training Session

1. **ANF is AI-Ready Storage**
    - Multi-protocol (NFS, SMB, Object REST API)
    - Enterprise-grade performance for AI workloads
    - Zero data movement integration with Azure AI services
2. **Object REST API Unlocks New Scenarios**
    - S3-compatible access to existing enterprise data
    - Direct integration with Fabric, AI Search, Databricks
    - No ETL required - same data, dual access protocols
3. **Agentic SelfOps is the Future**
    - AI agents managing their own infrastructure
    - Instant snapshots for safety before risky operations
    - Autonomous capacity and performance management
4. **"Better Together" Value Proposition**
    - ANF + Fabric + Data Agents = Democratized analytics
    - ANF + AI Search + OpenAI = Enterprise RAG

- ANF + Foundry Agents = Autonomous DataOps

Demo Flow Suggestions

1. **Start with the Problem**
    - "Enterprise data is locked in file shares"
    - "AI services need object/API access"
    - "Data movement is expensive and risky"
2. **Show the Solution**
    - Enable Object REST API on existing ANF volume
    - Create OneLake shortcut in under 5 minutes
    - Query data with natural language via Data Agent
3. **Highlight Enterprise Features**
    - Snapshot before any AI operation
    - Cross-region replication for BCDR
    - Cool access tiering for cost optimization

Appendix A: API Reference Quick Links

- ANF Object REST API
- ANF Management REST API
- azure-mgmt-netapp PyPI
- OneLake Shortcuts
- Fabric Data Agents
- Azure AI Search Python SDK
- Azure AI Foundry Agent Service
- azure-ai-projects PyPI

Appendix B: Sample Architecture Diagrams

(Mermaid diagram placeholders for draw.io/Visio conversion)

```
mermaid
```

```
graph TB
    subgraph "Scenario 1: ANF → OneLake → Fabric Data Agents"
        A[Data Engineers<br/>NFS/SMB] --> B[Azure NetApp Files<br/>Volume]
        B --> C[Object REST API<br/>S3 Bucket]
        C --> D[On-Prem Data Gateway]
        D --> E[Microsoft OneLake<br/>S3 Shortcut]
        E --> F[Fabric Lakehouse]
        F --> G[Fabric Data Agent]
        G --> H[Natural Language<br/>Query Results]
    end
```

mermaid

```
graph TB
    subgraph "Scenario 2: ANF → Azure AI Search → RAG"
        A[Document Store<br/>NFS/SMB] --> B[Azure NetApp Files]
        B --> C[Data Factory Sync]
        C --> D[Azure Blob Storage]
        D --> E[AI Search Indexer]
        E --> F[Vector Index]
        F --> G[Hybrid Search]
        G --> H[Azure OpenAI GPT-4o]
        H --> I[RAG Chat Response]
    end
```

mermaid

```
graph TB
    subgraph "Scenario 3: Foundry Agents → ANF SelfOps"
        A[User Request] --> B[Azure AI Foundry<br/>Agent]
        B --> C{Function Call}
        C --> D[Tool Executor<br/>Python]
        D --> E[azure-mgmt-netapp<br/>SDK]
        E --> F[ANF Control Plane<br/>REST API]
        F --> G[Azure NetApp Files<br/>Operations]
        G --> H[Snapshot/Clone/Resize]
        H --> I[Result to Agent]
        I --> J[Response to User]
    end
```